## OOP in Python

To map with real world scenarios, we started using objects in code. This is called object oriented programming.

note:-

Functions reduce redundancy and increase reusability

Redundancy ↓ reusability ↑

~~note: instance for class is object~~

## Class & object in python

class is a blueprint for creating objects.

```
# creating class
class student:
    name = "Avijeet Kumar"
# creating object instances
S1 = student()
print(S1.name)
```

## __init__ function

- constructor

All classes have a function called __init__(), which is always executed when the object is being initiated

```
# creating class
class student:
    def __init__(self, fullname):
        self.name = fullname

# creating objects
S1 = student("Avijeet")
print(S1.name)
```

note :- The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

attributes

all the data that is stored is called attributes.

class & instance attribute

class. attr.

obj. attr. obj. att > class attrib.

methods

methods are functions that belongs to objects.

# creating class

```
class student:
    def __init__(self, fullname):
        self.name = fullname
    def hello(self):
        print("hello", self.name).
```

# creating object

```
s1 = student("Ankit")
s1.hello()
```

that belongs to that specific object.

note :- self :- it is a instance of a class. it is used to access variables or methods

Q. create student class that takes name & marks of 3 subjects as arguments in constructor. then create a method to print the average.

```
class student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def get_avg(self):
        sum = 0
        for val in self.marks:
            sum += val
        print("hi", self.name, "The avg score is :", sum/7)

st = Student("Aniket", [97, 86, 92])
st.get_avg()
```

Static methods

Methods that don't use the self parameter (work at class level).

```
class student:
    @staticmethod # decorator
    def college():
        print("SNU college")
```

note :-

decorator

it allows us to wrap another function in order to extend the behaviour of the wrapped functions, without permanently modifying it.

Abstraction

hiding the implementation details of a class and only showing the essential features to the user. This reduce program complexity

Encapsulation

wrapping data and functions into a single unit (object) and restricting direct access to some of the object's component

Q create Account class with 2 Attributes - balance & account no., create methods for debit, credit & printing the balance

```
class account:

    def __init__(self, bal, acc):
        self.balance = bal
        self.accno = acc

    def debit(self, amount):
        self.balance -= amount
        print("rs", amount, "was debited")
        print("total balance =", self.get_balance())

    def credit(self, amount)
        self.balance += amount
        print("rs", amount, "was credited")
        print("total balance =" self.get_balance())

    def get_balance(self):
        return self.balance
acc1 = amount(10000, 12345)
acc1.debit(1000)
acc1.credit(500)
```

del keyword

used to delete object properties or object itself

eg:- del s1.name

del s1

private (like) attributes & methods.

Conceptual implementations in python

private attributes & methods are meant to be used only within the class and are not accessible from outside the class

Eg:-

```
class account
    def __init__(self, acc_no, acc_pass):
        self.acc_no = acc_no
        self.acc_pass = acc_pass

    def reset_pass(self):
        print(self.__acc_pass)

acc1 = Account("12345", "abcde")
print(acc1.acc_no)
print(acc1.reset_pass())
```

Note:- The internal function within the class will be able to show values, but no one from the outside can access them.

eg:-

```
class person
    __name = "anonymous"
    def __hello(self)
        print("hello person")
    def welcome(self)
        self.__hello()
p1 = person()
print(p1.welcome())
```

Class :- A blueprint or template that defines what the data and behaviour should look like.

Objects :- A specific instance created from that blueprint.

or

it is a real-time entity that represents a specific instance of a class and occupies memory at runtime.

or

A class acts as a blueprint, while memory is allocated only when object is created.

features of oops

it is designed to make code more modular, reusable and easy to maintain. These features allow developers to model real world scenarios effectively.

Note :- f"---{ }---"

f = formatted string.

The work of printing variables inside a string is done.

[redacted]: